

PROJECT REPORT OF CO-OP Project at Industry (Module-2)
ON
Adaptive Enemy AI Using Multi-Agent Reinforcement Learning in Action RPGs
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Prateek Khachi

Parth Kaushal

2210992071, 2210992030

Supervised By:

Dr. Gifty Gupta

Mentor

Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iv
	List of Figures and Tables	v
	Abstract	vi
1	Introduction	1
2	Methodology	3
3	Tools and Technologies	5
4	Implementation	7
5	Major Findings / Outcomes / Results	9
6	Conclusion and Future Scope	11
	References	13
	Appendices	14

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Adaptive Enemy AI Using Multi-Agent Reinforcement Learning in Action RPGs" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Gifty Gupta. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

(Prateek Khachi, Parth Kaushal)

Date: 18 May 2026

(2210992071 , 2210992030)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Gifty Gupta

Mentor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor and supervisor for their invaluable guidance and continuous support throughout this project. Their insights into reinforcement learning and game AI significantly shaped the direction of our research.

We are deeply thankful to the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology for providing us with the necessary resources, computational infrastructure, and academic environment that made this work possible.

We also extend our appreciation to all the participants who took part in the human evaluation sessions, providing honest and insightful feedback that helped validate our system's performance from a player experience perspective.

Finally, we acknowledge the open-source communities behind Gymnasium, Unity ML-Agents, and Stable Baselines3, whose tools formed the backbone of our implementation.

LIST OF FIGURES AND TABLES

Figures

Figure 1: Evolution of Game AI Techniques Across Decades

Figure 2: MARL Architecture Using CTDE

Figure 3: Comparative Win Rate Analysis

Figure 4: Adaptation Speed Comparison Using KL Divergence

Figure 5: Distribution of Emergent Cooperative Behaviors

Figure 6: Human Evaluation Results

Figure 7: Ablation Study Results

Figure 8: Performance Improvement from Scripted AI to MARL

Tables

Table 1: Evolution of Game AI Techniques Across Different Eras

Table 2: Comparative Performance of Scripted AI, Single-Agent PPO, and MARL

Table 3: Distribution and Frequency of Emergent Cooperative Behaviors

Table 4: Human Evaluation of Enemy AI Systems

Table 5: Ablation Study Results

ABSTRACT

The rapid evolution of artificial intelligence in digital gaming has significantly impacted player experiences, particularly in action role-playing games (RPGs), a market segment generating over \$15 billion in global revenue. Conventional enemy AI systems based on deterministic scripting methods, such as finite state machines and behavior trees, are predictable by nature, allowing experienced players to exploit repeating patterns and achieve win rates exceeding 90% after only a few encounters.

This project presents a comprehensive framework for developing adaptive enemy AI using Multi-Agent Reinforcement Learning (MARL), specifically the Multi-Agent Proximal Policy Optimization (MAPPO) algorithm within a Centralized Training with Decentralized Execution (CTDE) paradigm. A custom 2D combat simulation environment was designed using the Gymnasium and Unity ML-Agents packages to train 3 to 5 cooperative enemy agents over more than 1 million timesteps.

The system was evaluated through both quantitative and human-centric methods. Results demonstrated a 28% increase in adaptation speed as measured by KL divergence, and a 22% improvement in win rates over scripted baseline approaches. Emergent cooperative behaviors such as flanking and ambush strategies were observed in 65% of gameplay sessions. Human evaluations confirmed improvements in immersion (4.3/5), intelligence (4.1/5), and replayability (4.5/5). Despite computational cost challenges, the findings establish MARL as a transformative approach for game AI design.

Keywords: Multi-Agent Reinforcement Learning, Proximal Policy Optimization, Adaptive Game AI, Centralized Training Decentralized Execution, Emergent Behaviour, Action RPG Systems.

1. Introduction

The video game industry has witnessed rapid technological advancement, with action role-playing games (action RPGs) representing one of its most lucrative segments, valued at over \$15 billion globally. A critical component of the action RPG experience is the quality of enemy artificial intelligence, which directly governs player challenge, engagement, and long-term replayability. Despite the growing sophistication of modern games, enemy AI in most commercial titles continues to rely on deterministic scripting methods that are inherently predictable.

1.1 Problem Statement

Conventional enemy AI in action RPGs is primarily implemented using deterministic methods such as Finite State Machines (FSMs) or Behavior Trees (BTs). While these approaches are straightforward to implement and control, they suffer from a fundamental limitation: predictability. Empirical data indicates that players can identify and exploit repeating behavioral patterns after just 5 to 10 encounters, achieving win rates exceeding 90%. This creates a so-called 'glass ceiling' effect that sharply reduces the replay value and long-term engagement of a game.

1.2 Background and Motivation

Several AI paradigms have been explored for game development. Finite State Machines, used in titles like Dark Souls, represent behavior through discrete states but suffer from the 'state explosion problem' as complexity grows. Behavior Trees, used in The Witcher 3, offer hierarchical decision-making but require intensive manual tuning. Utility-based AI provides limited adaptability through weighted scoring. Single-agent reinforcement learning, as demonstrated by AlphaStar, shows strong adaptive capability but cannot model the group dynamics essential for RPG combat.

Multi-Agent Reinforcement Learning (MARL) overcomes these limitations by enabling groups of agents to learn cooperative, competitive, and emergent strategies simultaneously. Recent advances, particularly in the MAPPO algorithm, have demonstrated MARL's effectiveness in training up to 27 agents in complex environments. However, its application in small-scale action RPG combat scenarios of 3 to 5 agents remains underexplored.

1.3 Research Objectives

The primary objectives of this project are as follows:

- To design a MARL-based framework for adaptive enemy AI in action RPGs using the MAPPO-CTDE paradigm.
- To evaluate performance improvements in terms of win rate, adaptation speed, and emergent cooperative behaviors.
- To analyze player experience through human evaluation metrics including immersion, intelligence, and replayability.
- To assess the real-time deployment feasibility of MARL systems within game engine environments.

1.4 Scope of the Report

This report is structured as follows: Section 2 describes the methodology and MARL architecture. Section 3 covers the tools and technologies used. Section 4 details the implementation process. Section 5 presents major findings and results. Section 6 provides conclusions and future directions. References and appendices follow.

2. Methodology

The methodology employed in this project is grounded in Multi-Agent Reinforcement Learning theory and follows a systematic pipeline from environment design through training, evaluation, and deployment analysis. The framework adopts a Service-Oriented Architecture (SOA) layered into three tiers: Infrastructure (IaaS), Platform (PaaS), and Application (SaaS).

2.1 Environment Design

A custom two-dimensional combat arena of size 100×100 units was developed as the simulation environment. Within this arena, a single player agent interacts with a group of 3 to 5 cooperative enemy agents. Each agent operates on a 34-dimensional state space encoding player position, health, velocity, previous actions, and team-level attributes such as average health and spatial distribution.

The action space is discrete and includes: movement toward the player, light attacks, heavy attacks, evasive dodging, and coordination signals. This design achieves a balance between computational simplicity and behavioral complexity suitable for reinforcement learning experiments.

2.2 MARL Architecture: MAPPO-CTDE

The core of the proposed system is the Multi-Agent Proximal Policy Optimization (MAPPO) algorithm combined with the Centralized Training with Decentralized Execution (CTDE) paradigm. In this framework:

- Each enemy agent is equipped with an independent actor network operating on local observations, enabling autonomous decision-making during gameplay.
- During training, a centralized critic with access to the global environment state is used to stabilize learning and address the non-stationarity inherent in multi-agent systems.
- The PPO algorithm governs policy updates, ensuring they remain within a controlled range through a clipped objective function.

This combination allows agents to learn cooperative strategies during training while remaining independently executable at deployment time.

2.3 Reward Function Design

A custom multi-objective reward function was formulated to balance several competing goals: maximizing damage output on the player, maintaining team coordination and spatial distribution, surviving combat encounters, and penalizing predictable or repetitive behaviors. The predictability penalty, implemented through entropy-based regularization, is critical to prevent the learned policies from becoming exploitable by experienced players.

2.4 Training Pipeline

Training was conducted over 1 million timesteps on a cloud-hosted NVIDIA T4 GPU, with a total wall-clock training time of approximately 4.2 hours. Agents were trained against three distinct player strategy profiles: aggressive, defensive, and hit-and-run. Two baseline models were maintained for comparison: a rule-based scripted AI and a single-agent PPO model without inter-agent coordination.

2.5 Evaluation Methodology

The evaluation methodology extends beyond standard reward-based metrics to include:

- **Win Rate:** Overall performance comparison across player strategy types.
- **Adaptation Speed:** Measured using KL divergence between policy distributions over training episodes.
- **Emergent Behavior Analysis:** Identification and frequency tracking of cooperative tactics such as flanking, ambush, and feint strategies.
- **Human Evaluation:** Likert-scale surveys assessing immersion, perceived intelligence, and replayability across a group of player participants.

3. Tools and Technologies

3.1 Gymnasium

Gymnasium (formerly OpenAI Gym) served as the primary environment simulation framework. It provides a standardized interface for defining custom reinforcement learning environments, enabling consistent interaction between the agent and the simulated combat arena. The custom 100×100 unit combat environment was implemented as a Gymnasium environment class.

3.2 Unity ML-Agents

Unity ML-Agents Toolkit was used for game engine integration. It allows trained neural network policies to directly control agents within Unity game scenes, bridging the gap between the Python-based training pipeline and the real-time game engine execution environment. This tool was essential for validating the deployment feasibility of the trained MARL models.

3.3 Stable Baselines3

Stable Baselines3 provided reliable, well-tested implementations of reinforcement learning algorithms including PPO. It served as the base framework upon which the MAPPO extensions were built. Its modular design facilitated rapid experimentation with hyperparameter configurations during training.

3.4 PyTorch

PyTorch was the underlying deep learning framework used for defining, training, and updating the actor and critic neural networks. Its dynamic computation graph and extensive GPU support made it suitable for the iterative policy optimization required by the PPO algorithm.

3.5 Cloud Infrastructure: Google Colab & AWS

Training was performed on cloud-based GPU infrastructure, specifically NVIDIA T4 GPUs accessed via Google Colab. This approach leveraged the 'pay-per-use' paradigm to eliminate the need for permanent high-cost hardware, making large-scale MARL training accessible to smaller development teams.

3.6 Python

Python 3.x was the primary programming language used throughout the project for environment implementation, training pipeline scripting, data logging, and result visualization. Libraries including NumPy, Matplotlib, and Pandas supported numerical computation and analysis.

Tool / Technology	Version	Purpose
Gymnasium	0.28+	Environment simulation
Unity ML-Agents	2025 release	Game engine integration
Stable Baselines3	Latest	RL algorithm implementations
PyTorch	2.x	Neural network training
Python	3.10+	Primary programming language
NVIDIA T4 GPU	Cloud	Training compute infrastructure

Table 3.1: Summary of tools and technologies used in the project.

4. Implementation

4.1 Combat Environment Implementation

The combat arena was implemented as a custom Gymnasium environment class. The environment initializes with a configurable number of enemy agents (3 to 5) and a single player agent whose behavior is scripted using one of three strategy profiles. At each timestep, the environment receives actions from all active agents, updates positions and health values, computes rewards, and checks for terminal conditions such as player defeat or all enemies eliminated.

The 34-dimensional state vector for each agent includes: the agent's own position (x, y), health, velocity vector, the player's position and health, relative distances and angles to teammates, average team health, and a history vector of the last two actions taken. This rich observation space ensures agents can make contextually informed decisions.

4.2 Actor and Critic Network Architecture

Each agent's actor network is a three-layer fully connected neural network with ReLU activations. The input layer accepts the 34-dimensional local observation. Two hidden layers of 256 units each process the input, and the output layer produces a probability distribution over the 6 discrete actions via a softmax activation.

The centralized critic shares the same base architecture but accepts the concatenated global state, representing observations from all agents simultaneously. Its output is a single scalar value function estimate used to compute advantages for the PPO update step.

4.3 Training Configuration

Key hyperparameters used during training were as follows:

- Learning rate: 3×10^{-4} with linear decay schedule
- Clip parameter (ϵ): 0.2
- Discount factor (γ): 0.99
- GAE lambda (λ): 0.95
- Entropy coefficient: 0.01
- Mini-batch size: 64

- Number of PPO update epochs per rollout: 10

Agents were trained in parallel using rollout collection across multiple environment instances. Experience buffers were accumulated over 2,048 timesteps before each policy update phase.

4.4 Baseline Models

Two baseline models were implemented for performance comparison. The first is a rule-based scripted AI where each enemy independently pursues the player, attacks when within melee range, and retreats when health drops below 30%. The second is a single-agent PPO model trained without any coordination mechanism, using only individual local observations and no shared critic.

4.5 Ablation Study Design

An ablation study was conducted to isolate the contribution of individual components within the MARL framework. Four variants were tested: the full MARL system, a variant with no coordination mechanism (no team coordination signals in the observation or reward), a variant using independent critics per agent instead of a shared global critic, and a variant with no entropy regularization penalty.

5. Major Findings / Outcomes / Results

5.1 Win Rate Performance

The MARL-based system achieved a mean win rate of 55% across all player strategy types, compared to 42% for single-agent PPO and 35% for the scripted baseline. This represents a 20 percentage point absolute improvement over scripted AI and a 13 percentage point improvement over single-agent RL.

Model	Aggressive	Defensive	Hit-and-Run	Average
Scripted AI	35%	28%	41%	35%
Single-Agent PPO	42%	36%	48%	42%
MARL (Proposed)	52%	49%	63%	55%

Table 5.1: Win rate comparison across player strategy types.

5.2 Adaptation Speed

Adaptation speed was quantified using KL divergence between policy distributions across training episodes. The scripted AI exhibited a KL divergence of 0.00, reflecting its static, non-adapting nature. Single-agent PPO achieved a KL divergence of 0.23, indicating moderate behavioral adjustment. The MARL system achieved a significantly higher KL divergence of 0.45, representing a 28% increase over single-agent PPO and confirming substantially faster adaptation to changing player strategies.

5.3 Emergent Behavior Analysis

Perhaps the most striking finding of this project was the emergence of complex cooperative behaviors that were never explicitly programmed. In 65% of all observed gameplay sessions, one or more of the following emergent tactics were identified:

- Flanking (41% frequency): Two or more agents coordinately approached the player from opposite or oblique angles simultaneously, preventing effective blocking.
- Ambush (24% frequency): Enemy agents lay in wait or approached from outside the player's field of vision before converging.
- Feint (18% frequency): An agent performed a false attack approach to draw the player's attention, enabling teammates to strike from another direction.

The remaining 17% of observed behaviors included other forms of positional cooperation. These behaviors emerged purely from the reward structure and multi-agent interaction, without any hand-coded scripting.

5.4 Human Evaluation Results

Human evaluation was conducted using Likert-scale surveys (1–5) across three dimensions. Participants consistently rated the MARL-based AI significantly higher than the scripted baseline across all metrics:

Metric	Scripted AI	MARL System
Immersion	2.8 / 5	4.3 / 5
Intelligence	2.1 / 5	4.1 / 5
Replayability	2.9 / 5	4.5 / 5

Table 5.2: Human evaluation results comparing scripted AI and MARL system.

5.5 Ablation Study Results

The ablation study confirmed the criticality of each framework component. Removing the coordination mechanism reduced the win rate from 55% to 40% and emergent behavior frequency from 65% to 28%. Removing the shared critic reduced win rate to 48% and emergent behavior to 52%. Removing entropy penalties increased emergent behaviors to 71% but rendered the system highly exploitable by players who discovered its patterns, demonstrating a fundamental trade-off between behavioral diversity and robustness.

5.6 Real-Time Performance

The trained MARL system achieved a mean inference time of 16 milliseconds per decision step, which is well within the 16.67 millisecond threshold required to maintain 60 frames per second gameplay. This confirms the deployment feasibility of the proposed framework in real-time gaming contexts.

6. Conclusion and Future Scope

6.1 Conclusion

This project has demonstrated that Multi-Agent Reinforcement Learning is a viable and highly effective approach for developing adaptive enemy AI in action RPGs. By employing MAPPO within the CTDE paradigm and a carefully designed multi-objective reward function, the system produced enemy agents capable of dynamic, cooperative, and emergent behavior far beyond what deterministic scripting methods can achieve.

The key outcomes of this work include a 22% improvement in win rates over scripted baselines, a 28% faster adaptation speed as measured by KL divergence, and the emergence of sophisticated cooperative tactics in 65% of gameplay sessions. Human evaluations validated these improvements from a player experience perspective, with the MARL system scoring significantly higher on immersion, perceived intelligence, and replayability. The system also achieved real-time compatibility at 60 frames per second.

From a broader perspective, this research demonstrates a clear pathway away from rigid, rule-based game AI toward learning-based systems that grow more challenging and engaging over time. The accessibility of modern tools such as Gymnasium, Unity ML-Agents, and Stable Baselines3 means that this technology is within reach of independent developers and small studios, not just large commercial publishers.

6.2 Limitations

Several limitations must be acknowledged. The current implementation is confined to a simplified 2D combat environment, which does not capture the full spatial complexity, terrain variation, and vertical interactions found in 3D game worlds. The framework is optimized for small agent groups of 3 to 5 agents and may face coordination stability challenges at larger scales. The GPU training time of approximately 4.2 hours may also inhibit rapid iteration during development. Finally, while the system excels in tactical adaptability, it lacks narrative awareness, which is essential for creating fully immersive RPG experiences.

6.3 Future Scope

Several promising directions exist for extending this work:

- Extension to 3D environments using full game engine simulation with realistic terrain, physics, and vertical combat.
- Integration of hierarchical MARL to support both tactical (low-level combat actions) and strategic (high-level group decision-making) layers of behavior.
- Development of narrative-aware reward functions that align enemy behavior with story context and player progression.
- Exploration of curriculum learning to progressively increase training difficulty and improve sample efficiency.
- Investigation of federated or online learning approaches that allow AI to continue adapting to individual player styles after deployment.
- Application of the framework to other game genres including real-time strategy, survival, and battle royale formats.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, Cambridge, MA, USA: MIT Press, 2018.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, Jul. 2017.
- [3] C. Yu, A. Velu, E. Vinitsky, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," arXiv preprint arXiv:2103.01955, Mar. 2021.
- [4] OpenAI, "OpenAI Five," OpenAI, 2019.
- [5] Unity Technologies, "Unity ML-Agents toolkit," 2025.
- [6] A. Raffin, A. Hill, M. Ernestus, et al., "Stable Baselines3: Reliable reinforcement learning implementations," 2023.
- [7] FightLadder Benchmark, "A benchmark for competitive multi-agent fighting," 2024.
- [8] O. Vinyals, I. Babuschkin, W. M. Czarnecki, et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," DeepMind, 2019.
- [9] P. Hernandez-Leal, B. Kartal, and M. E. Taylor, "A survey of learning in multi-agent environments," ACM Computing Surveys, vol. 52, no. 4, pp. 1-35, 2019.
- [10] G. N. Yannakakis and J. Togelius, Artificial Intelligence and Games, Cham, Switzerland: Springer, 2018.
- [11] Farama Foundation, "Gymnasium documentation," 2023.
- [12] J. Foerster, N. Nardelli, G. Farquhar, et al., "Stabilising experience replay for deep multi-agent reinforcement learning," in Proc. ICML, 2017.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, Feb. 2015.
- [14] R. Lowe, Y. Wu, A. Tamar, et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," in Proc. NeurIPS, 2017.
- [15] T. Rashid, M. Samvelyan, C. Schroeder, et al., "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in Proc. ICML, 2018.

- [16] T. Wang, J. Wang, and C. Zhang, "Centralized training for decentralized execution in multi-agent reinforcement learning: A review," arXiv, 2020.
- [17] D. Silver, A. Huang, C. J. Maddison, et al., "Mastering the game of Go with deep neural networks and tree search," Nature, vol. 529, pp. 484-489, Jan. 2016.
- [18] G. Brockman, V. Cheung, L. Pettersson, et al., "OpenAI Gym," arXiv preprint arXiv:1606.01540, Jun. 2016.

APPENDICES

Appendix A: Hyperparameter Configuration Table

Hyperparameter	Value
Learning Rate	3×10^{-4} (linear decay)
Clip Parameter (ϵ)	0.2
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
Entropy Coefficient	0.01
Mini-batch Size	64
PPO Update Epochs	10
Rollout Length	2048 timesteps
Total Training Timesteps	1,000,000

Table A.1: Full hyperparameter configuration used during MARL training.

Appendix B: Observation Space Details

The 34-dimensional observation vector for each agent is composed of the following components:

- Own position (x, y): 2 dimensions
- Own health (normalized): 1 dimension
- Own velocity vector (vx, vy): 2 dimensions
- Player position relative to agent (dx, dy): 2 dimensions
- Player health (normalized): 1 dimension
- Distance and angle to each teammate (up to 4): 8 dimensions
- Average team health: 1 dimension
- Team spatial spread (standard deviation of positions): 2 dimensions
- Last two actions taken (one-hot encoded, 6 actions $\times$ 2): 12 dimensions
- Coordination signal received from teammates: 3 dimensions